

Práctica 2 - Modelos Probabilísticos, y Filtros Discretos

I-402 - Principios de la Robótica Autónoma

Prof. Dr. Ignacio Mas, Tadeo Casiraghi, Bautista Chasco

Modo de entrega: Enviar por el Aula Virtual del Campus todo el código comentado y los gráficos (.jpg ó .pdf), todo en una sola carpeta comprimida.

1. Muestreo de distribuciones de probabilidad

Implementar tres funciones en Python que generen muestras de una distribución normal $\mathcal{N}(\mu, \sigma^2)$. Los parámetros de entrada de dichas funciones son la media μ y la varianza σ^2 de dicha distribución. Como única fuente de aleatoriedad, utilizar muestras de una distribución uniforme.

1. En la primer función, generar las muestras de la distribución normal sumando muestras de 12 distribuciones uniformes.
2. En la segunda función, usar “muestreo con rechazo”
3. En la tercera función, usar el método de la transformación de Box-Muller. El método de Box-Muller permite generar muestras de una distribución normal usando dos muestras uniformemente distribuidas $u_1, u_2 \in [0, 1]$ según la ecuación:

$$x = \cos(2\pi u_1) \sqrt{-2 \log u_2} \quad (1.1)$$

Comparar los tiempos de ejecución de las tres funciones usando la función de Python `timeit`. Además, comparar el tiempo de ejecución de las tres funciones con el de la función `numpy.random.normal` de Python.

2. Path Planning

Los algoritmos de búsqueda en grafos como Dijkstra o A* pueden ser usados para planear caminos en grafos desde un lugar de inicio hasta un objetivo. Si las celdas de un mapa de grilla se representan como nodos conectados con sus celdas vecinas, estos algoritmos pueden aplicarse directamente para realizar planeamiento para robots. Para este ejercicio, consideramos las 8 celdas vecinas de una celda $\langle x, y \rangle$, que se definen como las celdas adyacentes a $\langle x, y \rangle$ horizontalmente, verticalmente y en diagonal.

Los archivos incluidos contienen una implementación de planeamiento en 2-D basado en grafos. El script `main.py` contiene el selector de algoritmo y es el que debe ejecutarse. Los ejercicios de esta sección se realizan implementando las funciones vacías o incompletas que acompañan al script principal.

2.1. Campos Potenciales Artificiales (APF)

Los campos potenciales modelan el entorno del robot como un sistema de fuerzas virtuales provenientes de la física. El objetivo ejerce una fuerza atractiva, mientras que los obstáculos del mapa generan fuerzas repulsivas.

1. Implementar la función `get_attractive_force` que calcule la fuerza hacia el objetivo utilizando un modelo lineal o cuadrático.
2. Implementar la función `get_repulsive_force`. Esta fuerza solo debe actuar si la distancia al obstáculo más cercano es menor que una distancia de influencia crítica d_0 .
3. Explicar conceptualmente el problema de los **mínimos locales** en este enfoque. Evalúe si esto ocurre para su implementación de campos potenciales.

2.2. Algoritmo de Dijkstra

El algoritmo de Dijkstra se usa para calcular caminos de costo mínimo dentro de un grafo. Durante la búsqueda, siempre se busca el nodo del grafo con el menor costo desde el punto de inicio y se agregan los nodos vecinos al grafo de búsqueda.

1. Sea $M(x, y)$ un mapa de grilla de ocupación. Durante la búsqueda, las celdas se conectan con sus celdas vecinas para construir el grafo de búsqueda. Completar la función `get_neighborhood` provista que define los vecinos de una celda. La función toma como entrada las coordenadas de una celda y el tamaño del mapa, y devuelve un vector de $n \times 2$ con las coordenadas de sus celdas vecinas, teniendo en cuenta los límites del mapa.
2. Implementar una función para los costos de un arco entre nodos que permita planear caminos de mínima longitud y libre de colisiones. Considerar la celda como un obstáculo si su probabilidad de ocupación supera cierto umbral. ¿Qué umbral se debería elegir? Implementar la función `edge_cost`.

3. Incluir información de ocupación en la función de costo que permita que el algoritmo elija celdas con baja probabilidad de ocupación sobre celdas con mayor probabilidad de ocupación.
4. Implementar el paso de actualización de costos. Para cada nodo *parent*, actualizar el costo de sus *childs* si el costo es menor a través de dicho *parent*.
5. ¿Cómo se ve afectada la velocidad de exploración del entorno a medida que aumenta el área a explorar? ¿En qué tipo de entornos resulta más eficiente Dijkstra: entornos abiertos con pocos obstáculos o entornos con muchos obstáculos? Justifique su respuesta considerando la cantidad de nodos que deben ser explorados.

2.3. Algoritmo A^*

El algoritmo A^* utiliza una heurística para realizar una búsqueda informada que resulta ser más eficiente que el algoritmo de Dijkstra.

1. ¿Qué propiedades debe tener dicha heurística para asegurar que A^* es óptimo?
2. Definir una heurística para planeamiento de robots móviles en 2-D. Completar la función `get_heuristic` provista. La función toma como entrada las coordenadas de una celda y del objetivo, y devuelve el costo estimado hasta el objetivo.
3. También complete la lógica de actualización como hizo en el punto 2.4
4. ¿Qué pasa si se aumenta la heurística usando h_2 , siendo h_2 un múltiplo de la heurística h definida en el punto anterior. Analizar el comportamiento con diferentes factores: $h_2 = \{1; 2; 5; 10\} * h$

2.4. Planificación de Cualquier Ángulo (Any-Angle): Theta*

Las grillas tradicionales restringen los movimientos a múltiplos de 45° . Theta* rompe esta limitación permitiendo que el padre de un nodo sea su “abuelo” si existe una línea de visión directa.

1. Implementar la actualización de costos en `update_costs_and_predecessors` evaluando las dos condiciones del algoritmo: el camino directo desde el abuelo (salto angular) frente al camino clásico de A^* .
2. Comparar visual y numéricamente la longitud real del camino final obtenido por A^* frente a Theta*. ¿Por qué Theta* logra trayectorias más cortas sobre la misma grilla?

3. Path Planning y Path Following en ROS

Ahora implementaremos lo que vimos de path planning en ROS, y le agregaremos path following para mover el robot. Para eso implementaremos dos paquetes de ROS 2. En ambos casos recuerde que tendrá que lanzar el mundo simulado de gazebo primero. Para eso se les provee un paquete nombrado *custom_map*. Una vez compilado, podrá lanzar el mundo en gazebo y rviz con el comando:

```
ros2 launch custom_map cafe.launch.py
```

3.1. Path Planning

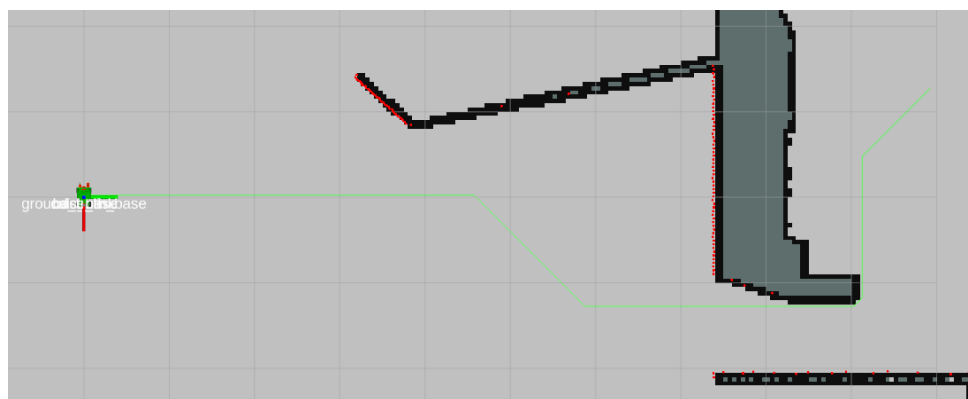
En el paquete *path_planning* verán el esqueleto de una implementación de path planning. Una vez que complete el código podrá lanzar el path planning de la siguiente manera:

```
ros2 run path_planning path_planner
```

Aspectos a importantes a notar:

1. La subscripción al mapa se hace con un mensaje de tipo **TRANSIENT_LOCAL**. Este tipo de mensaje se publica una o pocas veces, y cualquier nodo que se suscriba recibirá la información más reciente, sin importar que esta se haya publicado antes de que el nodo se suscriba.
2. La subscripción de odometria se hace con *calc_odom* ya que se utiliza la odometría que tendría el robot real.
3. Se proveen funciones de conversión de coordenadas de mapa a mundo y viceversa.
4. Si se genera el camino correctamente se podrá graficar en RVIZ automáticamente gracias al código provisto.

Deberán implementar algún algoritmo de path planning para generar el camino (guardado en *path* como una lista de tuplas de marcadores **en coordenadas de mapa**)



Una vez implementado su algoritmo vainilla responda las siguientes preguntas (no hace falta que modifique el código en base a sus respuestas):

1. ¿Es el camino calculado físicamente realizable? ¿Tiene este algoritmo toda la información necesaria para asegurar que el camino esta libre y que el robot podría efectivamente pasar por donde se sugiere pasar?
2. ¿Tiene en cuenta su algoritmo la orientación inicial del robot?

3.2. Path Following

En el paquete *path_following* verán el esqueleto de una implementación de path following. Una vez que complete el código podrá lanzar el path following de la siguiente manera:

```
ros2 run path_following pure_pursuit_node
```

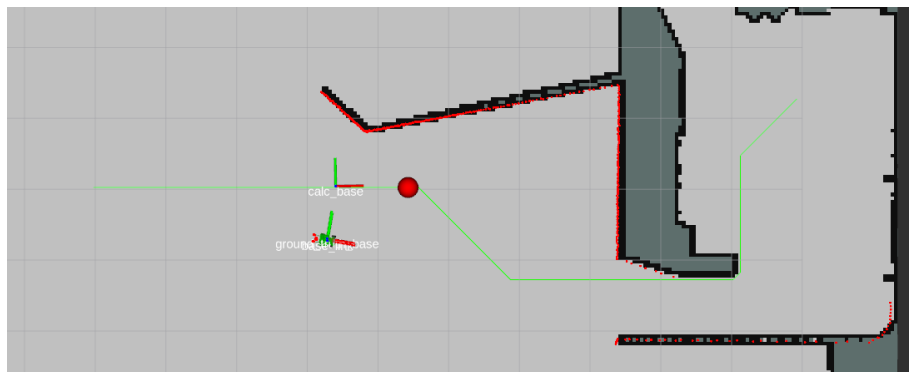
Aspectos a importantes a notar:

1. Ya está implementada la función de conversión de cuaternión a yaw
2. Deberán experimentar con distintos valores de:
 - a) `lookahead_distance`
 - b) `target_speed`
 - c) `goal_tolerance`

Prueben con criterio variaciones de estos parámetros para evaluar el desempeño del algoritmo.

3. Nuevamente el código se suscribe a la odometría calculada.
4. La función *publish_lookahead_marker* recibe una posición en coordenadas globales y publica un marcador para poder verlo en rviz.

Deberán implementar un algoritmo de pure pursuit para mover el robot y que siga el camino.



Una vez implementado su algoritmo responda las siguientes preguntas (no hace falta que modifique el código en base a sus respuestas):

1. ¿Cómo afecta el lookahead distance al desempeño del robot? ¿Qué sucede si es muy pequeño? ¿Qué sucede si es muy grande?
2. ¿Llega el robot real al objetivo? ¿En caso que falle, porqué?
3. ¿Qué fuentes de error puede observar? ¿Son de planeamiento? ¿De ejecución? ¿De alguna otra fuente?

4. Modelo de movimiento basado en odometría

Implementar un modelo de movimiento basado en odometría siguiendo estos pasos:

4.1. Movimiento Diferencial

1. Crear una función en Python que implemente un modelo de movimiento basado en odometría diferencial. los tres argumentos de entrada deberán ser:

$$\mathbf{x}_{t-1} = \begin{pmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{pmatrix} \quad \mathbf{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \quad \alpha = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{pmatrix}, \quad (4.1)$$

donde $\mathbf{x}_{t-1}$ es la pose anterior del robot, $\mathbf{x}_t$ es la pose nueva del robot calculada con odometría obtenida por los sensores del robot y α son los parámetros de ruido del modelo de movimiento (varianza de la distribución). La función deberá devolver una posible pose $\mathbf{x}_t$ del robot teniendo en cuenta los posibles ruidos de medición.

La implementación de la función deberá incluir el error de obtención de la odometría. Usar los métodos de muestreo del ejercicio 1 para generar muestras aleatorias normalmente distribuidas para introducir ruido en el modelo de movimiento.

2. Evaluar el modelo de movimiento desarrollado generando 5000 muestras con los siguientes argumentos:

$$\mathbf{x}_{t-1} = \begin{pmatrix} 0,0 \\ 0,0 \\ 0,785 \end{pmatrix} \quad \mathbf{x}_t = \begin{pmatrix} 0,01 \\ 0,01 \\ 0,765 \end{pmatrix} \quad \alpha = \begin{pmatrix} 20,0 \\ 20,0 \\ 0,01 \\ 0,01 \end{pmatrix}, \quad (4.2)$$

Graficar las posiciones $(\mathbf{x}_t, \mathbf{y}_t)$ para las 5000 muestras en un solo gráfico.

4.2. Movimiento Holonómico

1. Crear una función en Python que implemente un modelo de movimiento basado en odometría holonómica. los tres argumentos de entrada deberán ser:

$$\mathbf{x}_{t-1} = \begin{pmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{pmatrix} \quad \mathbf{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \quad \alpha = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{pmatrix}, \quad (4.3)$$

donde $\mathbf{x}_{t-1}$ es la pose anterior del robot, $\mathbf{x}_t$ es la pose nueva del robot calculada con odometría obtenida por los sensores del robot y α son los parámetros de ruido del modelo de movimiento (varianza de la distribución). La función deberá devolver una posible pose $\mathbf{x}_t$ del robot teniendo en cuenta los posibles ruidos de medicion.

La implementación de la función deberá incluir el error de obtención de la odometría. Usar los métodos de muestreo del ejercicio 1 para generar muestras aleatorias normalmente distribuidas para introducir ruido en el modelo de movimiento.

2. Evaluar el modelo de movimiento desarrollado generando 5000 muestras con los siguientes argumentos:

$$\mathbf{x}_{t-1} = \begin{pmatrix} 0,0 \\ 0,0 \\ 0,785 \end{pmatrix} \quad \mathbf{x}_t = \begin{pmatrix} 0,01 \\ 0,01 \\ 0,765 \end{pmatrix} \quad \alpha = \begin{pmatrix} 0,2300 \\ 0,0012 \\ 0,0044 \\ 0,4300 \\ 0,033 \\ 0,6383 \end{pmatrix}, \quad (4.4)$$

Graficar las posiciones $(\mathbf{x}_t, \mathbf{y}_t)$ para las 5000 muestras en un solo gráfico.

4.3. Evaluación de modelos

Observando el resultado de ambos modelos, responda:

1. ¿Cómo se diferencian las morfologías (formas) de la distribución de samples para ambos casos?
2. ¿Puedo utilizar la distribución de robots holonómicos para un robot diferencial? ¿Cuál sería la desventaja?
3. ¿Puedo utilizar la distribución de robots diferenciales para un robot holonómico?

4. Los valores de alpha están calibrados para una velocidad de medición de odometría en particular. Manteniendo los valores de alpha, evalúe la consecuencia de hacer un muestreo de movimiento para el caso en que la odometría sea 20 veces más lenta. Para eso puede hacer una prueba donde el valor de $\mathbf{x}_t$ acumula 20 movimientos. Es decir, evalúe las distribuciones cuando $\mathbf{x}_t$ es:

$$\mathbf{x}_t = \begin{pmatrix} 0,2300 \\ 0,1576 \\ 0,3850 \end{pmatrix} \quad (4.5)$$

5. Filtro Discreto - Bayes

Queremos estimar la posición de un robot que se mueve en un mundo unidimensional acotado. El mundo tiene 20 celdas y el robot se encuentra inicialmente en la celda 10. El robot no puede salir del área especificada.

En cada paso, el robot puede ejecutar el comando ‘avanzar un paso’ o ‘retroceder un paso’. Este movimiento puede tener cierto error, por lo que el resultado puede no siempre ser el esperado. De hecho, el robot se comporta de la siguiente manera: al ejecutar ‘avanzar un paso’ el resultado será el siguiente:

- el 25 % de las veces, el robot no se moverá
- el 50 % de las veces, el robot avanzará una celda
- el 25 % de las veces, el robot se moverá dos celdas
- el robot en ningún caso se moverá en el sentido opuesto o avanzará más de dos celdas.

El modelo de ‘retroceder un paso’ es similar, pero en el sentido opuesto. Ya que el mundo es acotado, el comportamiento al intentar ‘avanzar un paso’ en los bordes es el siguiente:

- si el robot está en la última celda, al tratar de avanzar se quedará en la misma celda el 100 % de las veces
- si el robot está en la penúltima celda, al tratar de avanzar se quedará en la misma celda el 25 % de las veces, y se moverá a la próxima celda el 75 % de las veces.

Lo mismo sucederá en sentido contrario en el otro extremo del mundo al intentar retroceder.

1. Implementar un filtro de Bayes discreto en Python y estimar la posición final del robot (belief) después de ejecutar 9 comandos consecutivos de ‘avanzar un paso’ y luego 3 comandos de ‘retroceder un paso’. Graficar el belief resultante de la posición del robot. Comenzar con el belief inicial de

```
bel = numpy.hstack((numpy.zeros(10),1,numpy.zeros(9))).
```

2. ¿Cómo queda la distribución luego de hacer 1000 pasos de avanzar un paso?¿Porqué sucede eso?
3. ¿Cómo queda la distribución luego de repetir 1000 veces '1 paso de avance y un paso de retroceso' ?¿Porqué sucede eso?